# System Design Masterclass Vol. 1: Low-Level Design (LLD) — OOP Foundations, SOLID Principles, and Core GoF Design Patterns in Java

Abstract

A definitive, practice-oriented engineering masterclass on Low-Level Design (LLD) and Machine Coding in Java, designed specifically for a 30-day intensive mastery curriculum (Days 1–7). This volume bridges the gap between academic theory and real-world software craftsmanship by eliminating abstract mathematical bloat and focusing entirely on actionable principles: the four OOP pillars from a clean-code perspective, rigorous before-and-after refactoring across all five SOLID principles, and complete runnable Java reference implementations for the top seven Gang of Four (GoF) design patterns (Strategy, Observer, Factory, Decorator, Thread-Safe Singleton with Double-Checked Locking, Builder, and State). It concludes with a structured 90-minute machine coding interview execution framework and an actionable daily 2-hour study schedule.

Index Terms

binary trees, algorithms, data structures, computational complexity, tree traversal, recursive algorithms, formal analysis

## 1 The 30-Day System Design Master Roadmap (2 Hours/Day)

Mastering System Design in 30 days requires an aggressive, structured, and practical approach. Instead of reading theoretical treatises, this 30-day curriculum allocates exactly 2 hours each day (60 hours total), divided equally between **Low-Level Design (LLD / Machine Coding in Java)** and **High-Level Design (HLD / Distributed Systems Architecture)**.

TABLE 1
The 30-Day System Design Intensive Curriculum (2 Hours / Day)

| Days | Module / Volume | Core Focus & Key Topics | Daily Commitment |
|---|---|---|---|
| Days 1–7 | Vol. 1: LLD Core Foundations | OOP Pillars, SOLID Principles, Top 7 GoF Patterns with runnable Java implementations. | 2 hrs / day (14 hrs) |
| Days 8–15 | Vol. 2: LLD Machine Coding & Concurrency | Multi-threading (‘ReentrantLock‘, ‘ExecutorService‘), Parking Lot, BookMyShow, Splitwise, LRU Cache. | 2 hrs / day (16 hrs) |
| Days 16–22 | Vol. 3: HLD Distributed Primitives | Load Balancers, Consistent Hashing, Caching (Redis), SQL vs NoSQL, Sharding, Kafka, CAP/PACELC. | 2 hrs / day (14 hrs) |
| Days 23–30 | Vol. 4: HLD Enterprise Case Studies | TinyURL, WhatsApp/Chat, Twitter Feed, Uber Geohash, Amazon Flash Sale, 45-min Interview Framework. | 2 hrs / day (16 hrs) |

This document constitutes **Volume 1: Low-Level Design (LLD) Core Foundations**, covering Days 1 through 7.

## 2 The Four OOP Pillars: An Interview & Clean Code Perspective

Every candidate can recite textbook definitions of OOP. In top-tier machine coding rounds (Amazon, Flipkart, Uber, Swiggy), interviewers evaluate how you apply these pillars to produce maintainable, extensible Java code.

### 2.1 Encapsulation: Invariant Protection, Not Just Getters/Setters

Encapsulation is not merely wrapping fields in ‘private‘ and generating getters and setters. True encapsulation guarantees that an object maintains its internal invariants and business rules.

```
// Bad: Leaky encapsulation (anemic model)
public class BankAccount {
   public double balance; // Anyone can set negative balance directly
```

```
}

// Good: Strict encapsulation with validated state transitions
public class BankAccount {
    private double balance;

    public BankAccount(double initialDeposit) {
        if (initialDeposit < 0) throw new IllegalArgumentException("Initial deposit >= 0");
        this.balance = initialDeposit;
    }

    public synchronized void deposit(double amount) {
        if (amount <= 0) throw new IllegalArgumentException("Deposit amount must be > 0");
        this.balance += amount;
    }

    public synchronized void withdraw(double amount) {
        if (amount <= 0 || amount > this.balance) {
            throw new IllegalStateException("Insufficient funds or invalid withdrawal amount");
        }
        this.balance -= amount;
    }

    public double getBalance() { return balance; }
}
```

### 2.2 Abstraction: Programming to an Interface

Abstraction hides implementation complexity behind stable contracts (‘interface‘ or ‘abstract class‘). In machine coding, you must always program to interfaces. This decouples the caller from the underlying provider, making services interchangeable.

### 2.3 Inheritance vs. Composition: Why Composition Wins

A classic junior mistake is creating deep inheritance hierarchies (‘Vehicle -> LandVehicle -> WheeledVehicle -> Car -> SportsCar‘). Deep hierarchies lead to fragile base class problems, code duplication, and tight coupling.

The Golden Rule of LLD: Favor object composition over class inheritance.

- Inheritance (IS-A): Use only when true behavioral substitution holds under all circumstances.
- Composition (HAS-A): Combine independent behaviors via injected components. If a car needs an engine, a ‘Car‘ should have an ‘Engine‘, not inherit from ‘Engine‘.

### 2.4 Polymorphism: Eliminating Brittle If-Else Chains

Polymorphism allows treating different underlying objects through a uniform interface. In clean LLD, polymorphism is your primary weapon for eliminating ugly ‘switch‘ and ‘if-else‘ chains.

## 3 Core Java OOP Language Mechanics & Interview Essentials

While the four pillars provide the architectural theory, Java interviews frequently probe deep into the language-specific OOP mechanics that govern how the JVM executes object-oriented code.

### 3.1 1. Abstract Class vs. Interface in Modern Java (Java 8 to 21)

- State vs. Behavior: An abstract class can define instance variables (mutable state) and constructors. An interface cannot hold mutable instance state (all fields are implicitly public static final).
- Multiple Inheritance: A Java class can implement multiple interfaces, but can extend only one abstract class (avoiding the diamond problem of multiple state inheritance).
- Modern Java Interfaces (Java 8+): Interfaces can define default methods (providing backward-compatible method bodies) and static utility methods. Java 9 added private interface helper methods.
- When to use which: Use an interface to define a peripheral capability or behavioral contract (Flyable, Serializable, Comparable). Use an abstract class when creating a tightly coupled family of classes that share common base code and mutable state (e.g., BasePizza, AbstractGameEngine).

### 3.2 2. The equals() and hashCode() Contract

One of the most frequently failed Java interview questions is explaining the strict mathematical contract between equals() and hashCode():

1) If two objects are equal according to equals(Object), their hashCode() must produce the identical integer.
2) If two objects have the same hashCode(), they are not necessarily equal (hash collision).
3) If you override equals(), you must override hashCode().

What breaks if you fail this rule? If you insert a custom object into a HashSet or use it as a key in a HashMap, Java uses hashCode() to compute the bucket array index. If you override equals() but forget hashCode(), two logically identical objects land in completely different buckets! As a result, map.get(key) returns null, and set.contains(key) fails silently, causing catastrophic state corruption in production!

### 3.3 3. Compile-Time vs. Runtime Polymorphism

- Compile-Time (Static / Method Overloading): Methods share the same name in the same class but have different parameter signatures (type, count, or order). Resolved at compile-time by the compiler. Return type differences alone are not sufficient.
- Runtime (Dynamic / Method Overriding): A subclass provides a specific implementation of a method declared in its parent class. Resolved at runtime by the JVM via Dynamic Method Dispatch using the virtual method table (vtable).

### 3.4 4. The final Keyword & The Immutable Class Pattern

In concurrent Java applications, immutability is the ultimate defense against race conditions. A class is truly immutable if its state cannot be modified after construction:

1) Declare the class final (prevents subclassing).
2) Make all fields private and final.
3) Do not provide setter methods.
4) Defensive Copying: If fields reference mutable objects (e.g., Date, List, Map), perform a deep copy in both the constructor and getters:

```
public final class ImmutablePosition {
    private final int x;
    private final int y;
    private final List<String> tags;

    public ImmutablePosition(int x, int y, List<String> tags) {
        this.x = x;
        this.y = y;
        this.tags = new ArrayList<>(tags); // Defensive copy in constructor!
    }

    public List<String> getTags() {
        return Collections.unmodifiableList(tags); // Defensive copy in getter!
    }
}
```

### 3.5 5. Java Access Modifiers Matrix

TABLE 2
Java Access Modifiers Scope Matrix

| Modifier | Same Class | Same Package | Subclass (Different Pkg) | World (Anywhere) |
|---|---|---|---|---|
| public | Yes | Yes | Yes | Yes |
| protected | Yes | Yes | Yes | No |
| default (Package-Private) | Yes | Yes | No | No |
| private | Yes | No | No | No |

### 3.6 6. Constructor Chaining: this() and super()

- this(...) invokes an overloaded constructor within the same class.
- super(...) invokes the direct superclass constructor.
- Rule: Either this() or super() must be the first line in a constructor body, ensuring the object hierarchy initializes from root Object down to the leaf instance.

### 3.7 7. Comparable vs. Comparator (Sorting & Ordering)

- Comparable<T> (Internal / Natural Ordering): Implemented by the domain class itself via compareTo(T o). Defines the single canonical sort order.
- Comparator<T> (External / Custom Strategy): Implemented as a standalone class or lambda (a, b) -> a.val - b.val. Allows defining multiple independent sorting strategies (e.g., sort by price, sort by name, sort by date) without altering the entity.

```
// Natural ordering: by rating
public class Player implements Comparable<Player> {
    private final String name;
    private final int elo;

    public Player(String name, int elo) { this.name = name; this.elo = elo; }
    public int getElo() { return elo; }
    public String getName() { return name; }

    @Override
    public int compareTo(Player other) { return Integer.compare(this.elo, other.elo); }
}

// Custom strategy: sort alphabetically by name
Comparator<Player> byName = Comparator.comparing(Player::getName);
```

### 3.8 8. Fail-Fast vs. Fail-Safe Iterators

- Fail-Fast (ArrayList, HashMap, HashSet): Maintain an internal modCount counter. If the collection is structurally modified (item added or deleted) while an iterator is traversing it, the iterator immediately throws ConcurrentModificationException. Rule: To remove during iteration, always use iterator.remove(), never list.remove()!
- Fail-Safe (ConcurrentHashMap, CopyOnWriteArrayList): Operates on a snapshot or clone of the underlying collection. Structural mutations do not throw exceptions, guaranteeing safe multi-threaded iteration at the cost of slight memory overhead.

## 4 The SOLID Principles Masterclass (With Real Java Refactoring)

The SOLID principles are the ultimate design guidelines for writing decoupled, testable, and enterprise-grade software.

### 4.1 S — Single Responsibility Principle (SRP)

Rule: A class should have one, and only one, reason to change.

Anti-Pattern (The God Object):

```
// Bad: Invoice does business math, database persistence, and PDF generation!
public class Invoice {
    private List<Item> items;

    public double calculateTotal() { /* business math */ return 0.0; }
    public void saveToDatabase() { /* JDBC/SQL persistence logic */ }
    public void printPdfReceipt() { /* PDF rendering logic */ }
}
```

Refactored Solution (Strict SRP):

```
public class Invoice {
    private final List<Item> items;
    public Invoice(List<Item> items) { this.items = items; }
    public double calculateTotal() {
        return items.stream().mapToDouble(Item::getPrice).sum();
    }
}

public class InvoiceRepository {
    public void save(Invoice invoice) {
        // Responsible ONLY for database persistence
    }
```

```
}

public class InvoicePrinter {
    public void print(Invoice invoice) {
        // Responsible ONLY for presentation / printing
    }
}
```

### 4.2 O — Open/Closed Principle (OCP)

Rule: Software entities should be open for extension, but closed for modification.

Whenever you add a new business requirement (e.g., adding UPI or Crypto payments), you should **never modify existing, tested code**. Instead, add a new class implementing a shared interface.

Refactored Java Solution (Strategy Pattern):

```
public interface PaymentStrategy {
    void pay(double amount);
}

public class CreditCardPayment implements PaymentStrategy {
    public void pay(double amount) { System.out.println("Paid " + amount + " via Credit Card"); }
}

public class UpiPayment implements PaymentStrategy {
    public void pay(double amount) { System.out.println("Paid " + amount + " via UPI"); }
}

// Open for new payment methods without changing PaymentProcessor
public class PaymentProcessor {
    public void processPayment(PaymentStrategy strategy, double amount) {
        strategy.pay(amount);
    }
}
```

### 4.3 L — Liskov Substitution Principle (LSP)

Rule: Subtypes must be substitutable for their base types without altering program correctness.

If class $S$ is a subclass of class $T$, any code expecting an instance of $T$ must work seamlessly with an instance of $S$ without throwing unexpected runtime exceptions or violating invariants.

The Classic Anti-Pattern: Flying Bird vs Penguin:

```
// Bad: Penguin violates the contract of Bird!
public class Bird {
    public void fly() { System.out.println("Flying..."); }
}

public class Penguin extends Bird {
    @Override
    public void fly() {
        throw new UnsupportedOperationException("Penguins cannot fly!"); // Violates LSP!
    }
}
```

Refactored Solution:

```
public interface Bird {
    void eat();
}

public interface FlyingBird extends Bird {
    void fly();
}

public class Sparrow implements FlyingBird {
```

```
    public void eat() { System.out.println("Eating seeds"); }
    public void fly() { System.out.println("Flying high"); }
}

public class Penguin implements Bird {
    public void eat() { System.out.println("Eating fish"); }
    // No fly() method -> LSP perfectly preserved!
}
```

### 4.4 I — Interface Segregation Principle (ISP)

Rule: Clients should not be forced to depend upon interfaces that they do not use.
Break "fat" interfaces into smaller, role-specific interfaces.
Refactored Solution:

```
// Bad: Fat interface forces unnecessary implementations
public interface Worker {
    void work();
    void eat();
    void sleep();
}

// Good: Segregated role-based interfaces
public interface Workable { void work(); }
public interface Feedable { void eat(); }

public class HumanWorker implements Workable, Feedable {
    public void work() { System.out.println("Human working"); }
    public void eat() { System.out.println("Human eating lunch"); }
}

public class RobotWorker implements Workable {
    public void work() { System.out.println("Robot assembling parts 24/7"); }
    // No need to implement eat()!
}
```

### 4.5 D — Dependency Inversion Principle (DIP)

Rule: High-level modules should not depend on low-level modules; both should depend on abstractions.
Refactored Solution (Dependency Injection):

```
public interface MessageSender {
    void send(String recipient, String message);
}

public class EmailSender implements MessageSender {
    public void send(String recipient, String message) {
        System.out.println("Sending Email to " + recipient + ": " + message);
    }
}

public class SmsSender implements MessageSender {
    public void send(String recipient, String message) {
        System.out.println("Sending SMS to " + recipient + ": " + message);
    }
}

// High-level service depends on abstraction (MessageSender), not concrete EmailSender
public class NotificationService {
    private final MessageSender sender;

    // Injected via constructor
    public NotificationService(MessageSender sender) {
```

```
        this.sender = sender;
    }

    public void notifyUser(String user, String alert) {
        sender.send(user, alert);
    }
}
```

## 5 Top 7 GoF Design Patterns for Machine Coding Interviews

Of the 23 Gang of Four (GoF) patterns, these 7 appear in 95% of all machine coding interview rounds.

### 5.1 1. Strategy Pattern (Behavioral)

Intent: Defines a family of algorithms, encapsulates each one, and makes them interchangeable at runtime. Common Interview Scenarios: Discount engines, payment processing, route planning (fastest, cheapest, scenic), sorting strategies.

### 5.2 2. Observer Pattern (Behavioral)

Intent: Defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified automatically. Common Interview Scenarios: Stock market ticker, YouTube subscriber alerts, Cricbuzz live cricket score updates, in-memory event buses.

```
import java.util.*;

public interface Observer {
    void update(String score);
}

public interface Subject {
    void register(Observer o);
    void unregister(Observer o);
    void notifyObservers();
}

public class CricketScoreBoard implements Subject {
    private final List<Observer> observers = new ArrayList<>();
    private String currentScore;

    public void setScore(String score) {
        this.currentScore = score;
        notifyObservers();
    }

    public void register(Observer o) { observers.add(o); }
    public void unregister(Observer o) { observers.remove(o); }
    public void notifyObservers() {
        for (Observer o : observers) { o.update(currentScore); }
    }
}
```

### 5.3 3. Factory & Factory Method Pattern (Creational)

Intent: Defines an interface for creating an object, but lets subclasses decide which class to instantiate. Decouples object instantiation from business logic. Common Interview Scenarios: Vehicle allocation in a Parking Lot, database driver connections, UI widget renderers.

```
public interface Vehicle { void park(); }
public class Car implements Vehicle { public void park() { System.out.println("Car parked"); } }
public class Bike implements Vehicle { public void park() { System.out.println("Bike parked"); } }

public class VehicleFactory {
    public static Vehicle createVehicle(String type) {
        if ("CAR".equalsIgnoreCase(type)) return new Car();
```

```
        if ("BIKE".equalsIgnoreCase(type)) return new Bike();
        throw new IllegalArgumentException("Unknown vehicle type: " + type);
    }
}
```

### 5.4 4. Decorator Pattern (Structural)

Intent: Attaches additional responsibilities to an object dynamically at runtime without modifying the underlying class (alternative to subclassing). Common Interview Scenarios: Pizza toppings (Base Pizza + Extra Cheese + Mushroom), Coffee maker, Java I/O streams (‘BufferedReader(new FileReader())‘).

```
public interface BasePizza { double cost(); }
public class Margherita implements BasePizza { public double cost() { return 150.0; } }

public abstract class PizzaToppingDecorator implements BasePizza {
    protected final BasePizza pizza;
    public PizzaToppingDecorator(BasePizza pizza) { this.pizza = pizza; }
}

public class ExtraCheese extends PizzaToppingDecorator {
    public ExtraCheese(BasePizza pizza) { super(pizza); }
    public double cost() { return pizza.cost() + 40.0; }
}

public class Jalapeno extends PizzaToppingDecorator {
    public Jalapeno(BasePizza pizza) { super(pizza); }
    public double cost() { return pizza.cost() + 30.0; }
}

// Client: BasePizza myPizza = new Jalapeno(new ExtraCheese(new Margherita()));
// Total Cost = 150 + 40 + 30 = 220.0
```

### 5.5 5. Thread-Safe Singleton Pattern (Creational)

Intent: Ensures a class has only one instance and provides a global access point to it. Interview Requirement: You must write the **Double-Checked Locking** version using ‘volatile‘ to prevent CPU instruction reordering.

```
public class DatabaseConnectionPool {
    // volatile is MANDATORY to prevent partially constructed object leakage
    private static volatile DatabaseConnectionPool instance;

    private DatabaseConnectionPool() {
        // Prevent reflection instantiation attack
        if (instance != null) throw new RuntimeException("Use getInstance() method");
    }

    public static DatabaseConnectionPool getInstance() {
        if (instance == null) { // First check (no lock)
            synchronized (DatabaseConnectionPool.class) {
                if (instance == null) { // Second check (with lock)
                    instance = new DatabaseConnectionPool();
                }
            }
        }
        return instance;
    }
}
```

### 5.6 6. Builder Pattern (Creational)

Intent: Separates the construction of a complex object from its representation, allowing the same construction process to create different representations. Interview Requirement: Solves the "Telescoping Constructor" anti-pattern when an object has 5+ optional configuration parameters.

```
public class UserProfile {
    private final String userId;     // Required
    private final String email;      // Required
    private final String phone;      // Optional
    private final int age;           // Optional

    private UserProfile(Builder b) {
        this.userId = b.userId;
        this.email = b.email;
        this.phone = b.phone;
        this.age = b.age;
    }

    public static class Builder {
        private final String userId;
        private final String email;
        private String phone = "";
        private int age = 0;

        public Builder(String userId, String email) {
            this.userId = userId;
            this.email = email;
        }

        public Builder phone(String phone) { this.phone = phone; return this; }
        public Builder age(int age) { this.age = age; return this; }
        public UserProfile build() { return new UserProfile(this); }
    }
}
// Usage: UserProfile user = new UserProfile.Builder("101", "pavan@work.com").age(28).build();
```

### 5.7 7. State Pattern (Behavioral)

Intent: Allows an object to alter its behavior when its internal state changes. The object will appear to change its class. Common Interview Scenarios: Vending Machine, Order Delivery Lifecycle (Placed → Paid → Shipped → Delivered → Returned).

### 5.8 8. Command Pattern (Behavioral) — Undo/Redo & Decoupled Actions

Intent: Encapsulates a request as an object, thereby letting you parameterize clients with different requests, queue or log requests, and support reversible operations (Undo/Redo). Critical Interview Use Cases: Chess Engine move history (Undo/Redo), transactional database rollbacks, graphical editor operations.

```
import java.util.*;

public interface Command {
    void execute();
    void undo();
}

public class MovePieceCommand implements Command {
    private final Board board;
    private final Position from;
    private final Position to;
    private Piece capturedPiece;

    public MovePieceCommand(Board board, Position from, Position to) {
        this.board = board; this.from = from; this.to = to;
    }

    @Override
    public void execute() {
```

```
        this.capturedPiece = board.getPiece(to);
        board.move(from, to);
    }

    @Override
    public void undo() {
        board.move(to, from); // Reverse move
        if (capturedPiece != null) board.placePiece(to, capturedPiece); // Restore captured piece
    }
}

public class CommandHistory {
    private final Deque<Command> undoStack = new ArrayDeque<>();
    private final Deque<Command> redoStack = new ArrayDeque<>();

    public void executeCommand(Command cmd) {
        cmd.execute();
        undoStack.push(cmd);
        redoStack.clear(); // Clear redo stack on new action
    }

    public void undo() {
        if (!undoStack.isEmpty()) {
            Command cmd = undoStack.pop();
            cmd.undo();
            redoStack.push(cmd);
        }
    }
}
```

## 6 The 90-Minute Machine Coding Interview Execution Framework

In top tech interviews, you are given 90 minutes to design and implement a fully functioning, compilable low-level system in your IDE. Follow this exact timed blueprint:

TABLE 3
The 90-Minute Machine Coding Time Management Blueprint

| Time Interval | Phase | Action Items & Key Deliverables |
|---|---|---|
| 0 – 10 mins | Requirement Discovery | Clarify functional scope, identify boundary constraints, list core actors. |
| 10 – 25 mins | Class Diagram & Schema | Design domain entities (‘models‘), interfaces, and service contracts. |
| 25 – 65 mins | Clean Implementation | Code models, strategies, factories, and service business logic in Java. |
| 65 – 75 mins | Concurrency & Validation | Add thread-safety (‘synchronized‘/‘ReentrantLock‘) and input validations. |
| 75 – 90 mins | Driver Code & Testing | Write a rich ‘Main.java‘ demonstrating all use cases and edge cases. |

### 6.1 Production Error Handling & Custom Exception Hierarchy

Interviewers severely penalize candidates who return null or boolean flags (return false;) when business rules fail. Production machine coding requires a structured unchecked exception hierarchy:

```
// Base domain exception (Unchecked RuntimeException)
public abstract class DomainException extends RuntimeException {
    public DomainException(String message) { super(message); }
}

public class ResourceNotFoundException extends DomainException {
    public ResourceNotFoundException(String resource, String id) {
        super(resource + " not found with ID: " + id);
    }
}

public class InvalidOperationException extends DomainException {
```

```
    public InvalidOperationException(String message) { super(message); }
}

// In service logic:
// if (spot == null) throw new ResourceNotFoundException("ParkingSpot", spotId);
// if (!spot.isFree()) throw new InvalidOperationException("Spot already occupied!");
```

## 7 Daily 2-Hour Action Plan for Days 1 to 7

- Day 1 (2 hrs): Read Section 2 (OOP Pillars). Implement the Encapsulated 'BankAccount' and composition examples in your IDE.
- Day 2 (2 hrs): Master Single Responsibility (SRP) and Open/Closed (OCP). Refactor an existing 'switch-case' discount calculator to the Strategy pattern.
- Day 3 (2 hrs): Master Liskov Substitution (LSP), Interface Segregation (ISP), and Dependency Inversion (DIP). Implement Dependency Injection for a notification engine.
- Day 4 (2 hrs): Code the **Strategy Pattern** and **Observer Pattern** from scratch. Build an in-memory Stock Price Notification simulator.
- Day 5 (2 hrs): Code the **Factory Method** and **Decorator Pattern**. Build a customizable Pizza billing calculator with dynamic toppings.
- Day 6 (2 hrs): Implement the **Thread-Safe Double-Checked Locking Singleton** and the **Builder Pattern**. Test against multi-threaded access.
- Day 7 (2 hrs): Implement a mini-project (e.g., A Vending Machine or State-driven Order Pipeline) combining Factory, Strategy, and State patterns.